KU LEUVEN

Labo 1 - CUDA

Robin De Vos (MEIECFS)
Anthony Tacquet (MEISWE)
Isaak Vervenne (MEISWE)

Februari 2026

1 Wat is er gerealiseerd?

In dit labo hebben we code geschreven om alle items in een array, omgekeerd terug te geven. Elke thread in onze 1 dim block zal een verschillende waarde nemen uit array a en deze op de omgekeerde positie op slaan in array b.

Onze code houdt rekening met mogelijke offset problemen mocht er een configuratie verschillend van een enkele block met n threads gebruikt worden.

```
1  __global__ void flip(int *a, int *b, int n) {
2    int idx = threadIdx.x + blockIdx.x * blockDim.x;
3    if (idx < n) {
4      b[idx] = a[n - idx - 1];
5    }
6  }
```

Listing 1: Code voor het verwisselen van de items in een array

2 Problemen

Deze opgave ging vrij vlot, bij de tweede compile werkte de code al, we hadden een kleine fout gemaakt bij het indexen van de array waarbij we -1 vergeten waren. Dit probleem hebben we uiteindelijk snel opgelost.

3 Vragen

3.1 What is the largest possible block size (max. threads per block)?

Dit is afhankelijke van de gpu architectuur en voor huidige gpu's is dit 1024 threads per block.

4 Appendix

4.1 Volledige code

```
  #include <iostream>
#include <cuda_runtime.h>

// CUDA kernel to add two arrays
__global__ void flip(int *a, int *b, int n) {
    int idx = threadIdx.x + blockIdx.x * blockDim.x;
    if (idx < n) {
        b[idx] = a[n - idx - 1];
    }
}

int main() {
    const int N = 9;
    int a[N], b[N];
    int *d_a, *d_b;

    // Initialize host arrays
    for (int i = 0; i < N; i++) {
        a[i] = i;
    }

    // Allocate device memory
    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));

    // Copy data from host to device
    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice
       );

    // Launch kernel (1 block of N threads)
    flip<<<1, N>>>(d_a, d_b, N);

    // Copy result back to host
    cudaMemcpy(b, d_b, N * sizeof(int), cudaMemcpyDeviceToHost
       );

    // Print results
    std::cout << "Original:␣\t";
    for (int i = 0; i < N; i++) {
        std::cout << a[i] << "␣";
```

```
39      }
40      std::cout << std::endl;
41      std::cout << "Result:␣\t";
42      for (int i = 0; i < N; i++) {
43          std::cout << b[i] << "␣";
44      }
45      std::cout << std::endl;
46
47      // Free device memory
48      cudaFree(d_a);
49      cudaFree(d_b);
50
51      return 0;
52  }
```

Listing 2: Volledige code